CS6493 Natural Language Processing
Assignment 02 Report

Chen Baizheng (72510800)

April 17, 2026

Question 1 (20 marks)

1) Padding and indexing in batched training

The vocabulary mapping is fixed by the assignment statement. After tokenization by whitespace:

- Sentence 1: `I like cats and dogs` → `[4, 7, 10, 17, 11]`
- Sentence 2: `She enjoys reading books and playing video games` → `[5, 8, 12, 13, 17, 18, 19, 20]`
- Sentence 3: `They play football every weekend` → `[6, 9, 14, 15, 16]`

The longest sequence length is 8, so right padding with 0 yields:

- Sentence 1 padded: `[4, 7, 10, 17, 11, 0, 0, 0]`
- Sentence 2 padded: `[5, 8, 12, 13, 17, 18, 19, 20]`
- Sentence 3 padded: `[6, 9, 14, 15, 16, 0, 0, 0]`

This shape alignment is required for vectorized mini-batch computation in embedding lookup and subsequent matrix operations.

2) Word embeddings

(a) Definition. A word embedding is a dense continuous vector representation $e_w \in \mathbb{R}^d$ for token w, where vector geometry (distance and direction) is optimized to encode distributional behavior.

(b) Advantage over one-hot. One-hot vectors are orthogonal and sparse, so all non-identical words are equally dissimilar under dot product. Embeddings support graded similarity and parameter sharing, which improves generalization to low-frequency and unseen contexts.

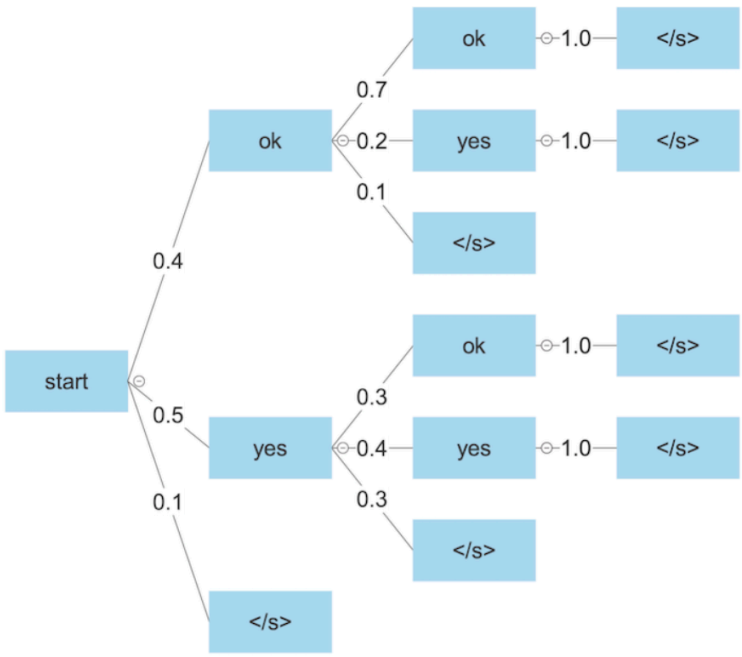

Figure 1: Decoding tree for Q1.3.

3) Greedy search vs beam search

Figure 1 gives the decoding tree.

(a) Greedy search. Greedy selects the locally largest probability at each step:

$$\texttt{start} \to \texttt{yes}(0.5) \to \texttt{yes}(0.4) \to \texttt{</s>}(1.0)$$

Path probability:

$$P_{\text{greedy}} = 0.5 \times 0.4 \times 1.0 = 0.2000.$$

Sequence: `yes yes </s>`.

(b) Beam search with $k = 2$.

- Step 1 (keep top-2): `yes` (0.5), `ok` (0.4).
- Step 2 expand all candidates:

$$\begin{aligned}
&\texttt{yes yes} : 0.5 \times 0.4 = 0.20, \quad \texttt{yes ok} : 0.5 \times 0.3 = 0.15,\\
&\texttt{yes \</s>} : 0.5 \times 0.3 = 0.15, \quad \texttt{ok ok} : 0.4 \times 0.7 = 0.28,\\
&\texttt{ok yes} : 0.4 \times 0.2 = 0.08, \quad \texttt{ok \</s>} : 0.4 \times 0.1 = 0.04.
\end{aligned}$$

- Keep top-2: `ok ok` (0.28) and `yes yes` (0.20).
- Final step appends `</s>` with probability 1.0 for both.

Best beam sequence is `ok ok </s>` with probability 0.2800.

(c) Why beam can outperform greedy. Greedy commits to a single local optimum early. Beam search preserves multiple high-probability prefixes, which reduces irreversible early mistakes and improves approximate global search quality.

4) BLEU calculation

Using BLEU-4 [2] with uniform weights $w_n = 1/4$:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\frac{1}{4}\sum_{n=1}^{4} \ln P_n\right)$$

where P_n is clipped n-gram precision and BP is brevity penalty.

From the code-based count verification:

$$P_1 = 0.8889, \quad P_2 = 0.7500, \quad P_3 = 0.7143, \quad P_4 = 0.6667.$$

Candidate length equals the closest reference length, so

$$\text{BP} = 1.0000.$$

Therefore,

$$\text{BLEU} = 0.7506.$$

Interpretation: lexical overlap is strong on unigrams/bigrams, while higher-order precision drops as expected due to phrase-level mismatch (e.g., `wooden table` vs `wooden chair`/`leather chair`).

Question 2 (20 marks)

1) Constraint analysis under $W^Q = W^K$ and $V = K$

Given

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V,$$

if $W^Q = W^K = W$, then

$$S = QK^\top = (XW)(XW)^\top.$$

So S has two key properties:

- Symmetry: $S_{ij} = S_{ji}$.
- Positive semidefinite Gram form.

Numerical verification from the reproducible code gives

$$\max_{i,j} |S_{ij} - S_{ji}| = 0.000000.$$

Directional limitation. Because S is symmetric, the mechanism cannot directly express asymmetric relational intensity (e.g., token i strongly depends on token j but not vice versa) at the score level.

Additional constraint $V = K$. Keys and values serve different roles (addressing vs transmitted content). Enforcing $V = K$ couples these roles, reducing representational flexibility and the model's ability to decouple retrieval criterion from output content space.

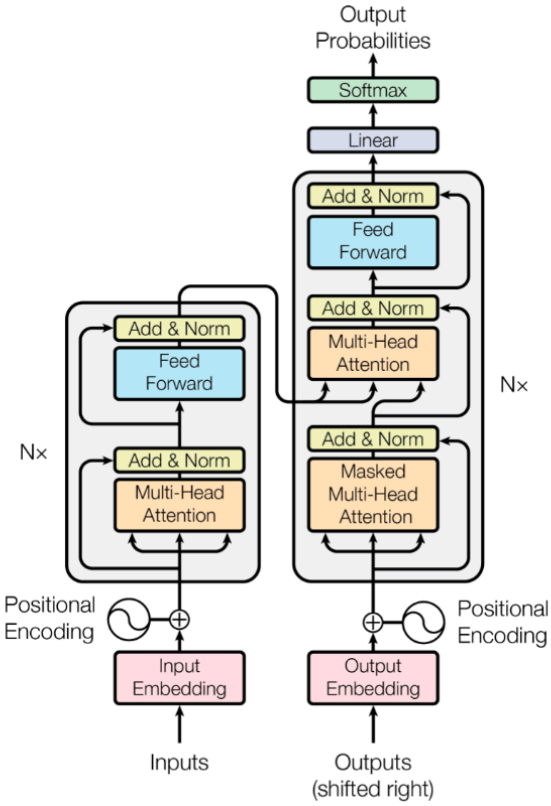

Figure 2: Transformer architecture [5].

2) Behavior as scaling coefficient β changes

For

$$\alpha_i = \frac{\exp(\beta q^\top k_i)}{\sum_j \exp(\beta q^\top k_j)}, \qquad c = \sum_i \alpha_i v_i,$$

we analyze two limits:

(i) $\beta \to 0$**.** A first-order expansion of softmax logits gives near-equal weights:

$$\alpha_i \to \frac{1}{n},$$

thus

$$c \to \frac{1}{n} \sum_i v_i,$$

which is the arithmetic mean of values.

(ii) $\beta \to \infty$**.** Softmax becomes a hard selection over maximal logit index:

$$\alpha \to \text{one-hot at } \arg\max_i q^\top k_i.$$

In the reproducible numeric example, argmax index is 1, so context converges to the corresponding value vector (or a mixture under exact ties).

3) Causal mask placement and explicit mask

Masked decoder attention uses

$$A = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right), \qquad O = AV.$$

Why add mask before softmax. Setting future logits to $-\infty$ before softmax ensures:

- masked positions receive exact probability 0,
- remaining valid positions are properly renormalized,
- gradients do not leak into forbidden future positions.

If zeroing happens after softmax, invalid positions already consumed probability mass and distort optimization.

For $n = 4$, the required causal mask is:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Question 3 (30 marks)

1) NLU vs NLG

NLU maps language input to meaning-level structures or task decisions; NLG maps internal representations/intent to fluent language output.

NLU examples:

- Sentiment classification.
- Extractive reading comprehension.

NLG examples:

- Machine translation.
- Abstractive summarization.

2) Classification type identification

- (a) Spam detection: **binary classification**.
- (b) Exactly one section from five classes: **single-label multi-class classification**.
- (c) Multiple applicable taxonomy labels: **multi-label classification**.
- (d) Ordered rating scale: **ordinal classification**.

3) Reading comprehension and SQuAD

(a) In SQuAD-style extractive QA, the model predicts the start/end positions of an answer span directly from the passage [4].

(b) Two reasons SQuAD was stronger than many earlier datasets:

- Scale and diversity: large number of QA pairs with realistic crowd-authored questions.
- Evaluation clarity: exact span supervision enables objective EM/F1 benchmarking.

(c) SQuAD 2.0 requires explicit answerability judgment (abstain/no-answer decision) with confidence calibration, not only span extraction [3].

4) Decoding and BLEU

(a) Greedy decoding chooses the maximum-probability next token at each step; drawback: myopic decisions and error accumulation.

(b) Exhaustive search is computationally infeasible because complexity grows exponentially with output length.

(c) Beam search keeps top-k partial hypotheses each step; k controls breadth/exploration. Length normalization reduces bias toward short hypotheses under product-of-probabilities scoring.

(d) BLEU components discourage different failure modes:

- n-gram precision discourages unrelated or hallucinated content.
- brevity penalty discourages overly short outputs that trivially achieve precision.

5) Two heuristic automatic metrics

- ROUGE [1]: overlap-based content coverage (often recall-oriented in summarization).
- BERTScore [6]: contextual semantic similarity using pretrained embedding alignment.

Question 4 (30 marks)

1) Campus assistant system design (15 marks)

System components.

- **Ingestion layer**: parse PDFs/web pages/policy docs, attach semester and timestamp metadata.
- **Knowledge processing**: clean, chunk, and index documents with source, section, version, and validity fields.
- **Retrieval layer**: hybrid sparse+dense retrieval for robust recall.
- **Reranking layer**: cross-encoder reranker for precision at top-k evidence.
- **Generation layer**: LLM prompt constrained by retrieved evidence, with mandatory citation format.

- **Safety layer**: confidence estimation, freshness checks, refusal policy, and escalation path to staff.

Query-to-answer workflow.

1. Receive and normalize student query (spelling, aliases, temporal terms).
2. Detect intent/domain (registration, deadline, grading, graduation).
3. Retrieve candidate passages from official indexed corpus.
4. Rerank and retain high-confidence evidence set.
5. Generate draft answer grounded only in retained evidence.
6. Run verification checks: citation coverage, contradiction check, policy freshness.
7. Return either (i) cited final answer, or (ii) abstention + clarification request + escalation hint.

2) Handling uncertain or missing information (15 marks)

(a) Why models still answer when uncertain. Autoregressive LLMs are optimized for next-token likelihood and usually instruction-tuned to be "helpful," so they preferentially produce fluent continuations even under weak evidence unless explicit abstention mechanisms are enforced.

(b) Two practical reliability strategies.

- **Answerability-gated refusal.** Define thresholds on retrieval confidence, reranker margin, and self-consistency score. If below threshold, output "insufficient evidence" and ask a targeted follow-up question.
- **Evidence-constrained generation with citation checks.** Force each key claim to map to retrieved passages; reject outputs with unsupported claims or stale-policy citations. This directly reduces hallucination and outdated responses.

Operational note. In production, both strategies should be monitored with refusal rate, correction rate, and user satisfaction to tune the safety-utility tradeoff.

References

[1] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

[2] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of ACL*, 2002.

[3] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for squad. *arXiv preprint arXiv:1806.03822*, 2018.

[4] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[6] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with bert. In *International Conference on Learning Representations*, 2020.